rogii-wellbore-geology-prediction

4th Place Solution

Rank	4
Team	L & J & A & A
Author	James Day
Topic ID	733480
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733480
Retrieved with Kaggle CLI on 2026-08-07.

Overview

Our solution is an ensemble of 3 independent approaches:

- **James** predicts TVT (true vertical thickness) from the surrounding topography and the drill's motion relative to the surface, then renders the GR (the gamma ray data) and topography data as a 2D image. CNNs predict offsets at eight control points to correct the initial topography-based path.
- **Lightsource** renders each well as a multichannel 2D canvas and uses U-Nets to trace the TVT path using an approach similar to dense image segmentation. A 1D Squeezeformer is used to produce complementary predictions.
- **Alijs** uses a particle filter to track plausible TVT paths from drill motion and GR matching, then improves it with topography, bidirectional decoding, and learned corrections. This reduces the wrong-layer matches that hurt simpler particle-filter solutions.

Ensemble weights were selected by fitting a ridge regression model on our OOF (out of fold) predictions. Only one of our three solutions performed well on the private leaderboard, but the other parts were useful in local cross validation and the full ensemble was robust enough for a cash gold finish.

Pipeline	Ensemble Weight	CV Score	Public LB	Private LB
James	0.644	5.143	5.463	5.730
Lightsource	0.327	5.570	5.897	7.045
Alijs	0.029	7.035	6.217	9.138
Full Ensemble	N/A	4.998	5.452	5.870

James's Part

Overview

My solution is inspired by the winner of the PhysioNet ECG digitization competition. I tried imitating the vision model architecture that previously beat my segmentation models. It worked well for this competition too.

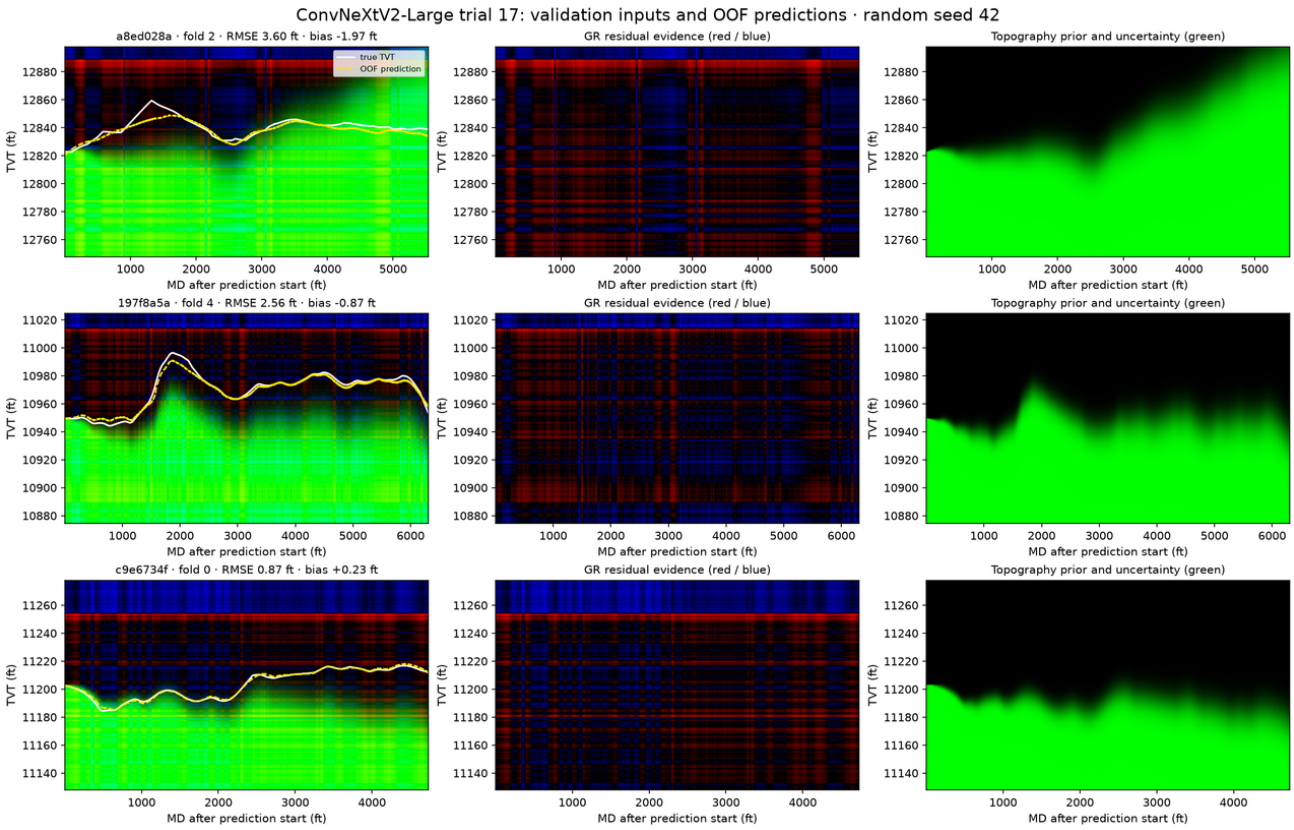
Predictions are generated in 4 main steps:

1. **Predict the slope of the rock formations along the drill path.** This is done by feeding X, Y, Z, azimuth, and pitch values to an ensemble of tabular regression models. It works because the training and test wells originated from the same basin with a mostly-linear global slope and deviations that can sometimes be predicted based on neighboring wells. The regression models produce uncertainty estimates so that downstream vision models can tell how confident the topography ensemble was about any particular well.
2. **Predict TVT based on the topography.** We're given a last-known TVT, so if change in TVT is known, then it becomes trivial to precisely calculate the unknown TVT values. Change in TVT = change in rock elevation - change in the drill's Z coordinate. All of the Z values are given, so the only real unknown is change in rock

elevation. The slope predictions from step #1 and the distance traveled by the drill are used to roughly fill in the blank.

3. **Render the topography predictions & GR data as a 2D image.** Each vertical column of the image describes the data at a particular measured depth, horizontal rows describe potential true vertical thicknesses. The red and blue image channels are populated with GR residual values describing if the observed GR value at the measured depth is higher or lower than what you'd expect if the TVT hypothesis is correct. Green represents the probability, according to the topography model, that the true TVT exceeds the candidate value.
4. **Predict topography offsets.** The rendered images are fed to an ensemble of ConvNeXt V2 Large models with feature pyramid regression heads, loosely based on "strategy 1" in the PhysioNet winner's writeup. These models predict 8 control point offsets to shift the topography-based TVT predictions up or down. Offsets between control points are linearly interpolated. Training occurred in 3 stages: pretraining on synthetic well data from one generator, then finetuning on synthetic well data from a second generator, then finetuning on the real well data.

A visualization of the vision model inputs overlaid with the final TVT predictions and true labels is included below.



Topography Prediction

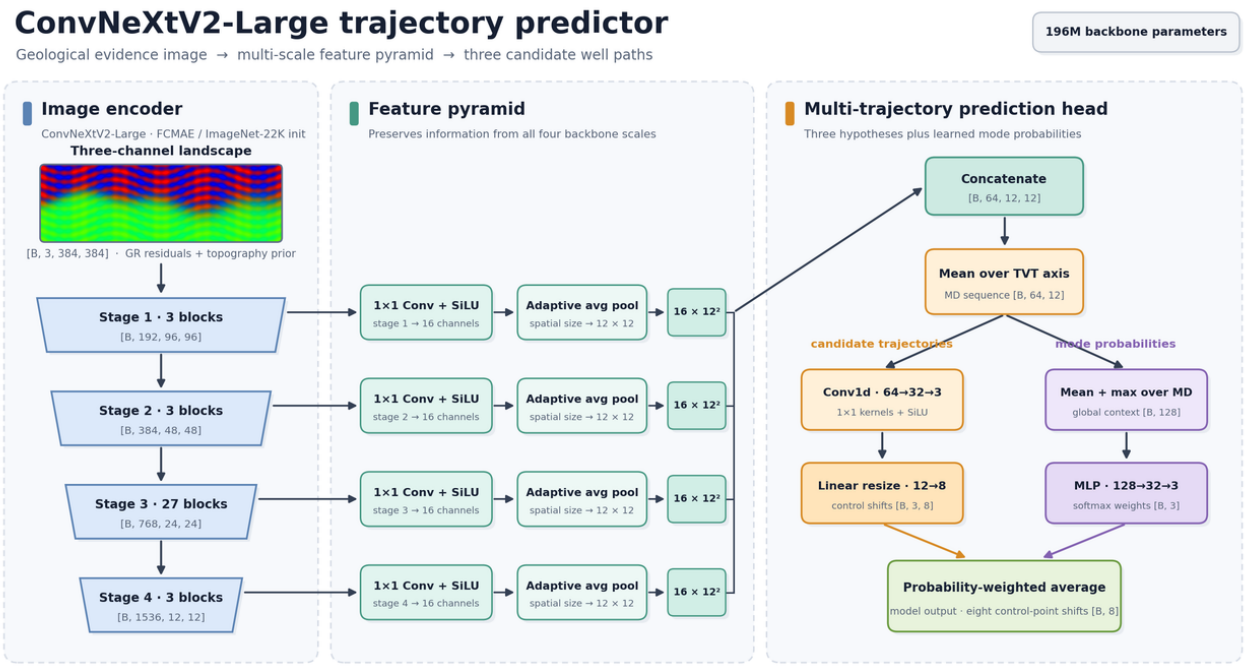
Rock formation slopes are predicted for 750-foot segments along the drill path.

~90% of the ensemble weight is devoted to vision models which receive predictions from relatively simple LightGBM quantile regression models that predict upper and lower confidence intervals for dA/dM (derivative of ANCC with respect to MD) based on the segment's midpoint coordinates and the direction the drill was headed (azimuth and pitch).

The other ~10% received predictions from a stronger ensemble of TabPFN, RealMLP, and LightGBM models which predict dA/dM, dA/dX, and dA/dY. This approach is substantially more accurate (e.g. CV slope RMSE 0.0100 -> 0.0088, private LB score without using the vision models 12.6 -> 10.8) and produces better calibrated confidence intervals (pearson correlation between predicted sigma and real error 0.31 -> 0.40), but wasn't used in conjunction with vision models trained on real well data because the CV gains of the topography-only predictions were ~20x larger than the public LB gains, and finetuning models on these predictions caused them to focus "too heavily" on the topography outputs, thereby making the public LB scores worse. Furthermore, the vision models are somewhat sensitive to changes in how the confidence intervals are calibrated (swapping in a strong topography ensemble for a vision model trained with a weaker topography model makes the results worse in CV).

As a result, in order to make my models focus more heavily on the GR data without data augmentations that hurt in CV, most vision models in my ensemble used the old topography. The only one that didn't was a pretrained foundation model trained solely on synthetic well data, without training on any wells from the real basin, thereby preventing it from "overfitting" to any particular topography model's behavior or learning to focus too heavily on the strong topography outputs.

Vision Model Architecture



All of the vision models in my ensemble used the exact architecture above. The diversity came from data augmentation configurations, learning rate schedule configurations, topography selection, data bagging, and including a pretrained foundation model (trained purely on synthetic data) without any real-well finetuning, not architectural variations.

Backbone choice:

- Small (~50M parameter) models seemed to be optimal with only 5K synthetic pretraining wells and only ~700 real wells for finetuning, but with small models there was little-no score improvement when scaling from tens of thousands of pretraining wells to hundreds of thousands. ConvNeXt Large with ~540k training wells works well enough that the small models are no longer useful, even for ensemble diversity purposes.

- ConvNeXt was the model family that worked best for me at small scales in early experiments (beating ViTs and EfficientNet V2), so it is the one I chose to scale first. Later, I managed to get stronger small model results with MambaVision, but didn't have time to scale it, and it wasn't strong enough to include in the ensemble. I underestimated the power of scaling because I did my ConvNeXt Large finetuning with a suboptimal configuration that wasn't fixed until the last 48 hours, so it wasn't as much of a priority as it should have been.

Other architectural choices:

- Many of the wells are ambiguous, where there are multiple plausible paths that match the GR data well. The competition scoring metric rewards models for predicting paths between the likely valid possibilities, but this is somewhat difficult for them to learn with a standard single-trajectory prediction head. As a result, loosely imitating "Direct Multi-Modal Inversion of Geophysical Logs Using Deep Learning" by Alyaev et al., I have the model predict 3 series of control point offsets and the probability of each of them being the one that's closest to the true path, then compute a weighted average of those paths. Optimizing purely for the MTP (multi-trajectory prediction) loss function proposed in that paper is somewhat suboptimal in terms of minimizing single-path RMSE, so the model is trained to minimize both of them (the MTP loss and weighted-average trajectory loss) as a form of multi-task learning (using a hybrid loss function).
- Similar to the PhysioNet competition winner's observations, I found it beneficial to use features from multiple spatial scales in the vision backbone, not just pooled features from the final layer. However, this workload seemed to benefit from an even more extreme form of that: using all 4 spatial scales instead of only 2.
- The choice of layers and pooling strategies used in the feature pyramid and MTP head was selected based on extensive trial and error with small models. Broadly speaking, it works best with relatively "shallow" heads with a small number of channels in their internal representations (forcing the backbone to do most of the work), and in some parts of the model (e.g. mode probability selection) doing both mean and max pooling then concatenating their outputs works better than pure mean or max pooling.

Synthetic Pretraining Data

My models were pretrained on 500k synthetic wells from my "V0" data generator followed by 37K synthetic wells from a filtered subset of my "V3" generator outputs. The foundation model's CV RMSE score, without finetuning on real wells, is 5.9 ft. In early experiments, pretraining on synthetic wells improved final accuracy by ~1.5 ft (8.2 -> 6.7 ft CV RMSE after real well training). I assume it matters even more after the refinements that have been made since then (almost certainly 2+ ft gain).

To quote Codex's summary of how the generators work:

> **Main steps in the V0/V0.1 generator:**
>
> 1. Sample the last-known TVT and prediction length from a bank of real wells.
> 2. Generate a synthetic typewell GR-versus-TVT window centered on the last-known TVT.
> 3. Generate an independent horizontal-well TVT-delta path from the empirical TVT path bank, then add the last-known TVT.
> 4. Interpolate the typewell at those TVT values to obtain expected horizontal-well GR.
> 5. Generate GR residuals conditioned on expected GR, including an expected-GR-dependent component plus low-, medium-, and high-frequency noise.

> 6. Form observed horizontal GR as: observed GR = expected GR + residual.
> 7. Sample a single BUDA slope and construct a linear BUDA path.
> 8. Back-calculate: Z = BUDA - TVT.
> 9. Write the known prefix row and prediction rows.
>
> V0's synthetic topography is relatively simple: piecewise-constant Gaussian slope errors are added to the true constant BUDA slope, with their scale calibrated to a requested integrated TVT RMSE.
>
> **Main steps in the V3 generator:**
>
> 1. Generate and regularize a synthetic typewell GR-versus-TVT window.
> 2. Choose a realistic well length and construct the MD grid.
> 3. Generate a BUDA path containing:
>   - A sampled starting level and global slope
>   - Correlated slope wander
>   - Explicit fault jumps
> 4. Generate Z conditioned on that BUDA path. The Z generator models dZ/dMD from the smooth BUDA increments plus correlated residual variation.
> 5. Derive TVT through the physical relation: TVT = BUDA - Z + relation_constant.
> 6. Resample BUDA/Z jointly if the resulting TVT path falls outside the generated typewell's TVT range.
> 7. Interpolate the typewell at the resulting TVT path to obtain expected GR.
> 8. Generate a residual conditional on that expected-GR spine.
> 9. Return observed GR as: observed GR = expected GR + residual.
> 10. Write the known prefix and prediction rows.
>
> V3's residual generator uses expected GR substantially more thoroughly than V0: expected level and local contrast influence both the mean residual and its local variance, followed by correlated fat-tailed noise, coherent excursions, and missingness.
>
> Its topography OOF generation is also empirical rather than globally Gaussian: predicted slope errors and uncertainties are sampled per segment conditional on the synthetic well's true BUDA geometry, roughness, slope changes, and real OOF behavior.
>
> So the concise distinction is:
>
> - V0: TVT → expected GR → residual/GR → BUDA → derive Z
> - V3: BUDA → conditioned Z → derive TVT → expected GR → residual/GR

My main role in generator development was torturing GPT-5.5 and Opus-4.8 with a ton of complaints about statistical discrepancies between the real and synthetic data. I didn't pay much attention to the implementation details, beyond the main high-level steps.

The V3 data was filtered by training an EfficientNet V2 Small model on 10k synthetic wells, finetuning it on real wells, then having both models make predictions for 50k additional synthetic wells and discarding the samples where the "error" of real-well model's predictions was over double the error of the model trained on synthetic wells.

By all metrics, the V3 data is more realistic than V0 (even without the filtering), but it's not necessarily "better". Models seem to converge more quickly when trained on the V0 data and the correlation between V0 training loss and real-well cross validation loss is quite good for up to 3 million training samples (e.g. 6 epochs over 500k wells or 3 epochs over 1m). Beyond 3M samples, ConvNeXt V2 Large starts to exhibit degenerate behavior: disregarding the "classification" part of the MTP loss entirely in order to get lower weighted average regression loss. That serves its training objective (minimizing the hybrid loss function), but hurts generalization to real wells, and I couldn't quickly find a balance of loss weights or hyperparameter tweaks that fixes it without side effects worse than the original problem.

Mamba seemed to benefit from mixing the pretraining datasets from each generator (roughly an equal mix), but for CNNs a staged V0 -> V3 approach worked better.

Vision Model Training Details

Shared architecture

All stages use:

- ConvNeXt V2-Large: convnextv2_large.fcmae_ft_in22k_in1k_384
- A feature-pyramid head using all four backbone stages
- 16 projected channels per pyramid level and 32 hidden channels
- Three candidate trajectories with MTP mode probabilities
- 384×384 rendered inputs
- ±75 ft TVT window
- No topography centering or ImageNet input normalization
- BF16 mixed precision
- AdamW; only learning rate is specified, so other settings use PyTorch defaults
- No model-head dropout

Pretraining predicts 12 control points; real-well finetuning reduces this to 8.

Stage 1 pretraining

- Data: 500k V0.1 synthetic wells
- Initialization: pretrained ImageNet/FCMAE ConvNeXt V2-Large weights
- Training: 6 epochs, configured batch size 16, two-process distributed training
- Optimization: OneCycleLR, cosine annealing, peak LR 2.053×10^{-4}, 15% warmup, momentum cycling
- Loss: Huber with 9.48 ft threshold
- MTP losses:
  - Best-trajectory regression
  - Classification weight: 0.164
  - Probability-weighted-average trajectory loss weight: 1.0
- Real-well validation uses the legacy, non-centered topography OOFs

- Only the first fold is trained; both best and final checkpoints are saved

The principal augmentation recipe is:

- Mixup: 75%, alpha 0.624
- Vertical/horizontal image flips: 50% / 25%
- GR-residual inversion and residual flip: 25% each
- Simulated vertical shift: 25%
- Typewell flip during simulated shifting: 25%
- Topography-error permutation: 25%
- Horizontal-well GR noise: 50%, 7.09% standard deviation
- Typewell GR noise: 25%, 8.98% standard deviation
- Vertical masking: 50%, two masks, up to 11.4% each
- Heavy augmentations active for the first 80% of training (vertical shifts & topography error permutation disabled later)

No cutmix, channel dropout, affine GR augmentation, or horizontal masking is enabled.

Stage 2 pretraining

Stage 2 keeps essentially the entire Stage 1 recipe unchanged:

- Initialization: Stage 1's best checkpoint
- Data: 37k filtered V3 synthetic wells
- Training: 6 epochs, batch size 16
- Same architecture, LR schedule, loss, augmentations, resolution, and TVT radius

Real-well finetuning ensemble

Trial	Role	Epochs / batch	OneCycle schedule	Loss	CV RMSE	Blend weight
0	Frozen Stage 2 baseline, V2 topography	N/A	N/A	N/A	5.9270	11.15%
1	Baseline finetune	64 / 8	Cosine, `1.027e-4`, 15% warmup	Huber 9.48	5.2940	16.81%
8	Affine-heavy variant	80 / 16	Linear, `3.699e-5`, 30% warmup	MSE	5.2842	3.20%
17	Best individual model	112 / 8	Cosine, `3.284e-5`, 15% warmup	Huber 0.549	5.2013	48.50%
25	Shorter, low-LR variant	48 / 16	Linear, `1.987e-5`, 10% warmup	Huber 3.013	5.2826	20.33%

The four actual finetunes use five-fold real-well CV, initialize from the Stage 2 best checkpoint, and use the legacy non-centered topography OOFs. Trial 0 is a manually injected frozen foundation model and instead uses the newer V2 topography blend (not a real Optuna trial).

Their main augmentation differences are:

- Trial 1: Reuses the Stage 1 recipe almost exactly - strong mixup and noise, vertical flips, and 80% heavy-augmentation duration.
- Trial 8: Replaces many flips/noise operations with strong horizontal-well and typewell GR affine transformations; also uses horizontal and vertical masks. Heavy augmentation lasts 50% of training.
- Trial 17: Uses augmentations throughout all 112 epochs. It emphasizes horizontal-well GR noise, typewell affine transformation, GR-residual flipping, simulated shifts, and topography-error permutation.
- Trial 25: Uses strong horizontal-well affine augmentation, milder typewell affine augmentation, horizontal flipping, GR inversion, simulated shifts, and relatively small vertical masks. Heavy augmentation lasts 50%.

The loss weighting also contributes diversity:

- Trial 1: MTP classification 0.164, weighted-average loss 1.0
- Trial 8: 0.0625, weighted-average 1.0
- Trial 17: 0.485, weighted-average 1.0
- Trial 25: 0.0898, weighted-average 0.5

After optimal TTA and quarter-point re-anchoring, the blend improves full OOF RMSE from trial 17's 5.2013 to 5.1428, a gain of 0.0585.

Test Time Augmentation

The final ensemble used 2 forms of test time data augmentation:

- **Vertical flip (0.13 CV improvement):** Unlike a traditional vertical flip augmentation (just flipping the image and generating predictions for both the flipped and non-flipped versions), this simulates what would happen if everything below ground (e.g. the series of rock layers described by the typewell GR data) was upside down and the drill was headed in the opposite vertical direction relative to the surface during image rendering. So, the topography data in the green channel still makes physical sense, with green pixels near the bottom of the image. Predictions for the "flipped" image are inverted and averaged with the standard orientation predictions.
- **Quarter re-anchoring (0.18 CV improvement):** Predictions are generated for the full prediction range, then the horizontal well data is cropped to just the last 75% of the prediction range and the image is re-rendered for the last 75% as if the prediction 25% of the way through the prediction range was the last "known" TVT value. Then, predictions are regenerated for the last 75% and averaged with the original predictions (using equal weight). In early experiments, doing a second re-anchor stage 50% of the way through the prediction range was also beneficial in CV, but hurt on the private LB, and adding a third re-anchor stage hurt in both CV and public LB, so only quarter point re-anchoring was used.

These augmentations generally helped on the leaderboard too, but I didn't do ablation testing for the models in my final ensemble, so I don't know the private LB impact.

GR residual inversion helped for some models, but not ConvNeXt V2 Large. Horizontal flips were consistently harmful.

Lightsource's Part

1. Approach

Predict TVT on the unknown MD suffix of a horizontal well from:

- known TVT prefix
- horizontal GR
- trajectory (MD, Z)
- vertical typewell (TVT, GR)

Physical prior:

```
observed_GR ≈ interp(typewell_GR, true_TVT) + noise
```

We build a geometric prior from the known prefix and trajectory, then learn a **residual correction**. Two model families:

1. **2D UNets** — segmentation in residual-TVT × MD space
2. **1D Squeezeformer** — sequence regression of the same residual along MD

Training is staged: **synthetic pretrain → post-pretrain → real-well finetune → NNLS blend**. At inference we decode autoregressively with **chunk reanchoring**.

---

2. 2D modelling

Model

Unet, encoder from scratch, 16 input channels, 1 output class.

Backbones in the final ensemble: **ResNet-34**, **ConvNeXt-Tiny**, **Swin-Tiny** (plus a ResNet-34 variant trained from a `geo_plane` synthetic recipe).

Canvas:

- full grid 384 × 512 (residual axis × MD)
- train crop 256 × 256
- inference: either tiled 256×256 or single-pass fullcanvas 384×512 (chosen per model on OOF)

Input channels (16)

GR–typewell mismatch, horizontal GR, residual axis, known/future masks, geometric prior features, trajectory (`z`, `dz`, `d2z`, `md`).

Target

Mode **`cum_residual`**: the network predicts a cumulative boundary mask — probability that the true residual TVT boundary is below each row.

Decode:

```
prob = sigmoid(logits)
row  = sum_over_H(prob) - 0.5
residual = interp(row → residual axis)
tvt = geometric_prior + residual
```

Loss

```
BCE + Dice/Jaccard + row_loss(decoded boundary) + optional smooth
```

---

3. 1D modelling

Model

Squeezeformer sequence model over the well MD.

Features: base 1D well signals + a GR residual profile over ±100 ft TVT offsets (201 offsets, strip CNN), so the network sees how GR matches the typewell as a function of hypothesized TVT.

Target

Dense **prior residual** along MD:

```
y = TVT_true − geometric_prior
```

Known prefix is masked; the model is scored mainly on the unknown suffix. Inference uses the same chunk-reanchor loop as 2D (`h_match=64`, reanchor 500 ft).

Loss

Dense regression on the residual sequence (Huber / Gaussian-NLL style), weighted toward the unknown suffix. Optional aux losses: smoothness, monotonicity, sparse control points.

---

4. Augmentations

Finetune focuses on GR ambiguity and prior error — the failure modes that matter at test time:

- empirical GR noise bank (residuals `GR − interp(typewell, TVT)` from train wells)
- GR dropout (random NaNs), gain / offset / white noise
- magnitude warp
- typewell swap
- prior shift / slope noise / slope breaks
- coherent time stretch / warp on GR, TVT, Z, MD together
- mixed full-canvas samples vs random crops
- hflip, vflip
- mixup
- channel dropout
- random known frac
- random crop

Prefix length is randomized (~25–65% known). Pretrain mostly relies on synthetic diversity rather than heavy online augs.

---

5. Pretrain

Honest synthetic pretrain: real MD/Z + typewell; synthetic TVT and GR.

Generation sketch:

1. sample known prefix length
2. build geometric prior from prefix + Z
3. corrupt prior sometimes
4. add residual (drift + spline + bumps), clamp `dTVT/dMD`
5. `GR = interp(typewell, TVT) + noise_bank`

---

6. Post-pretrain

After the shared pretrain we continue synthetic training per backbone: same UNet / Squeezeformer, weights resumed, optimizer reset, generation switched to `geo_plane`.

Unlike the residual-on-prior pretrain recipe, TVT is sampled from a geological depth surface:

```
geo_z(MD) = (TVT_prefix + Z)_PS + plane_slope·ΔMD + piecewise_knots(MD)
TVT       = geo_z − Z
```

Generation sketch:

1. sample known prefix length (anchor at PS)
2. set `geo_z_PS = TVT_prefix[PS] + Z[PS]`
3. sample a gentle MD slope + 4–6 piecewise-linear knots (zeroed on the known prefix, ramped into the unknown)
4. `TVT = geo_z − Z`; hard-set the known prefix; clamp `dTVT/dMD`; keep TVT in typewell range
5. build geometric prior from prefix + Z (optionally corrupt)
6. `GR = interp(typewell, TVT) + noise_bank`

Alone (no finetune on real wells) this synthetic stack already scored about **8.9 RMSE** on OOF. Finetune on real wells then starts from these checkpoints.

---

7. Reanchoring

Long unknown suffixes drift if decoded in one shot. We decode in MD chunks:

1. canvas / 1D window from current known TVT
2. predict the next ~`N` feet of MD (`N` typically 500–950 ft)
3. append those points to the known prefix → rebuild prior
4. repeat until the well ends

Original known prefix is hard-constrained. Same idea for 2D and 1D.

---

8. Blend

OOF predictions from diverse full-retrain members (ResNet / geo-plane ResNet / ConvNeXt / Swin / Squeezeformer-1D) are combined with NNLS

- seed checkpoints inside a member are averaged equally
- member weights come from NNLS on OOF
- inference uses each member's best canvas mode + reanchor stride

Ours alone ≈ **5.57** OOF RMSE.

Alijs's Part

I started this competition with a simple Particle Filter as a baseline. On top of that, a LightGBM residuals corrector was added, which gave quite a good boost on CV, but especially on the public LB. In general, this competition wasn't very friendly to an LGB-style approach, as it worked at the row level and was missing the whole well-level picture. The obvious next step was to add some NN-based solution at the well level. I started to dig in that direction with UNet and CNN models, but around that time I was invited to join a team. As all the

teammates already had strong NN-based approaches, I deliberately didn't continue with my NNs, instead focusing on pushing the LightGBM pipeline further for diversity in the team blend.

For some reason (which I'll most likely be digging through after a couple of days), both my single-model CV and public LB scores turned out to differ significantly from the private LB. Given that every new LightGBM feature I added usually improved some wells while hurting others, the most likely explanation is that the private LB turned out to contain significantly more of that type of wells my LightGBM wasn't strong on.

My final solution part is a particle-filter-plus-corrector pipeline for predicting the wellbore's stratigraphic position (TVT) along the lateral. The backbone is a 64-seed particle filter that matches the well's gamma-ray log against its typewell to track vertical position, run both forward and backward. On top of that, a LightGBM residual corrector (an L2+L1 blend) predicts the backbone's error from ~115 self-diagnostic features — PF seed spread, stride and derivative features, a Viterbi second-opinion decode with typewell-rung aliasing companions, forward/backward disagreement, and disagreement with a non-ML global stretch-squeeze DP decoder. Corrected predictions then pass through a projection/smoothing stage.

Overall, it was an interesting experience with a somewhat different type of data than I'm used to working with. Thanks to the organizers for this interesting competition. And of course, the biggest thanks to my team for the great teamwork and such a strong finish!

Arunodhayan's Part

SDF Variant

1. Approach

Predict **TVT** on the unknown MD suffix of a horizontal well as a **signed distance field** in residual-TVT × MD space:

- per pixel, the signed vertical distance to the true boundary
- the boundary is the zero level set of that field
- decoded to a TVT path by **soft expectation** over rows, not root-finding

```
sdf(row, col) = row_residual − true_residual(col)
```

Initialised from Lightsource's `geo_plane` post-pretrain weights.

---

2. Model

`smp.Unet` , **ResNet-34**, 16 input channels, 1 output class.

Canvas:

- full grid **384 × 512** (residual axis × MD)
- train on the **full canvas, no random crops**
- inference: single-pass fullcanvas, reanchor 500 ft

Target

Mode **`sdf_residual`** : per-pixel signed vertical distance (residual-axis units) from that pixel to the true boundary.

Decode:

```
p    = exp(-|sdf| / tau)          # tau ~ 0.08, per column
p   /= sum_over_H(p)
row  = sum_over_H(row_index * p)
residual = interp(row → residual axis)
tvt = geometric_prior + residual
```

Soft expectation, not the zero crossing. Zero-crossing decode is worth **+1.70** — it is unstable wherever the field is flat.

Loss

```
sdf_reg + sign + row_loss(decoded boundary)
```

Jaccard segmentation component, Gaussian-NLL row loss, MD-weighted tail emphasis. Finetune strengthens the **row** term as in 2D.

---

3. Training

Stage A — resume from Lightsource's `geo_plane` post-pretrain. Input channels are identical across modes, so encoder/decoder transfer directly; only the interpretation of the output image changes.

Stage B — finetune on real wells. 50 epochs, early stop on rolled-out validation RMSE, patience 8; stops at epoch 20–28 in practice.

---

4. Augmentations

Inherited from the finetune stack. Re-tested the ones we doubted:

aug	setting	tested
hflip	p = 0.35	off → **+0.299**; p = 0.7 → +0.020
vflip	p = 0.03	p = 0.10 → −0.038; p = 0.5 → **+0.325**
mixup	p = 0.06, α 0.80	p = 0.5 → **+0.154**
time stretch	p = 0.44, 0.90–1.11	—
time warp	p = 0.31, 5 knots, 0.11	—
magnitude warp	p = 0.32, 4 knots, 0.10	—
GR noise bank	p = 0.31	—
GR gain / noise / offset	0.09 / 1.40 / 7.75	—

aug	setting	tested
GR dropout	0.15–0.16	—
slope break	p = 0.19	—
typewell swap	p = 0.18	—
prior shift / slope noise	0.14 / p = 0.21	—
global TVT offset	p = 0.09, max 2.58	—
channel dropout	p = 0.32, rate 0.19	—
known frac	0.17–0.68	—

---

5. What worked

change	Δ
soft decode vs zero-crossing	**−1.70**
equal-weight ensembling	**−0.43**
full canvas, no crops	**−0.39** (9/10 folds, p = 0.011)
hflip	**−0.30** (4/5 folds, p = 0.028)
Optuna loss config	−0.16
`geo_plane` pretrain	−0.15 (3/5 folds)

Unselected recipe means **7.279 → 6.910** across 16 seeds.

6. What didn't

change	Δ
empirical prefix-fraction sampling	**+0.357**
vflip p = 0.5	**+0.325**
mixup p = 0.5	**+0.154**
hard-well mining (all thresholds)	+0.15 ⋯ +0.53
fitted ensemble weights	+0.079 / +0.122
clip TVT to typewell range	+0.29
train on longer unknown sections	+0.08
loss-weight / dropout / LR sweeps	~0

2 wins in ~22 five-fold experiments. Both changed what the model outputs.

---

7. Notes

Fold-0 trap. Four configs won on fold 0 and lost on the 5-fold mean. Mixup improved fold 0 by −0.112 while folds 2 and 4 each degraded by +0.38.

Noise floor is recipe-dependent. Run-to-run sd of the 5-fold mean: 0.103 early recipe, **0.192** final one.

Selection bias. Across 16 seeds, no seed beat the order-statistic prediction (best-of-8: predicted 6.6387, observed 6.6389).

Fitted weights lose. Nested leave-one-group-out over wells:

pool	in-sample	nested	equal
15 members	6.385	6.578	**6.456**
12 members	6.188	6.323	**6.244**

---

8. Result

12 members, equal weight: 8 seeds of the locked recipe plus small config variants (canvas, pretrain, flip/mixup probabilities). **6.244** OOF RMSE.